

Comment optimiser l'exécution d'une requête SQL à l'aide de l'algèbre relationnelle ?

EliottP

2025 – 2026

Sommaire

Les bases du traitement des requêtes SQL

Les optimisations naïves

Optimisation dépendante des données

Analyse des résultats

Conclusion

Ressource

Description du traitement d'une requête

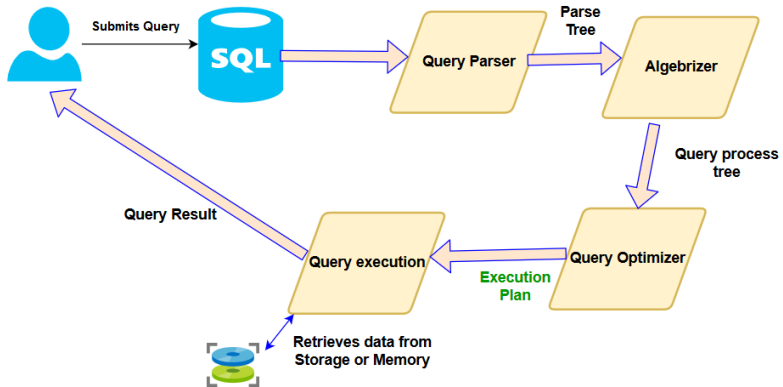


Figure: Traitement d'une requête SQL par un SGBD.

Source : Quest

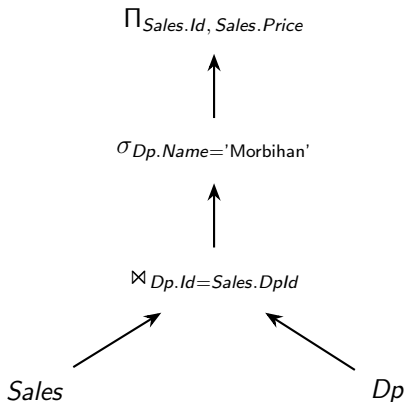
Edgar F. Codd (1970)

Calcul relationnel $\iff$ Algèbre relationnelle

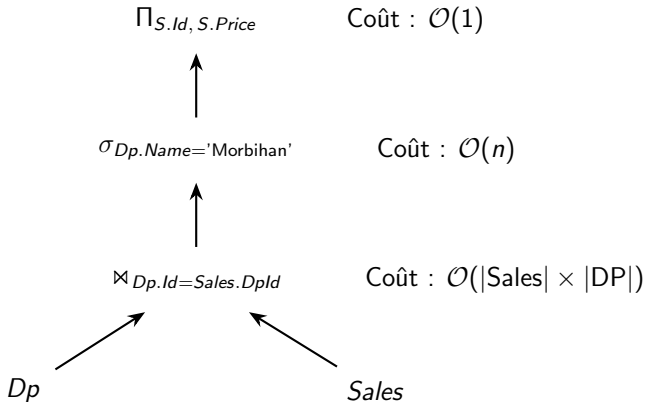
```

1 SELECT S.Id, S.Price
2 FROM Sales
3 JOIN Dp ON Dp.Id = S.DpId
4 WHERE Dp.Name = 'Morbihan';

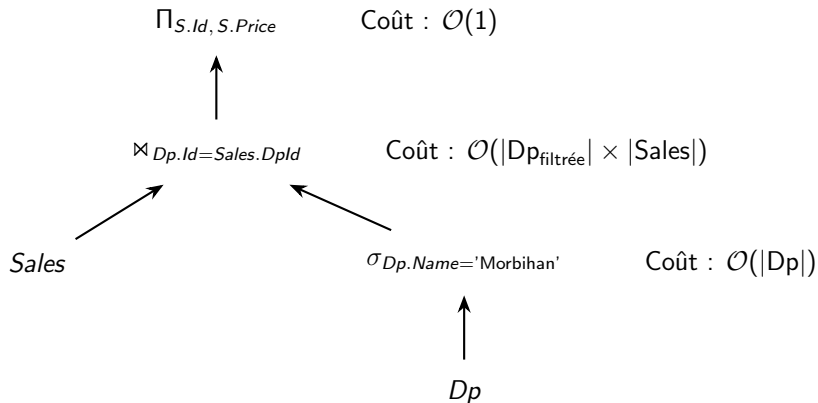
```

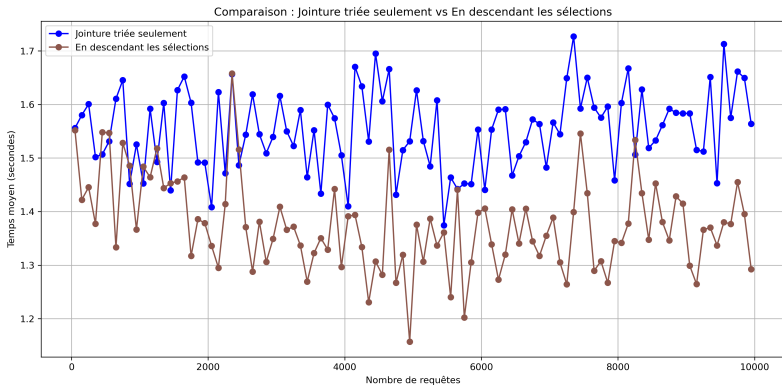


La descente des sélections

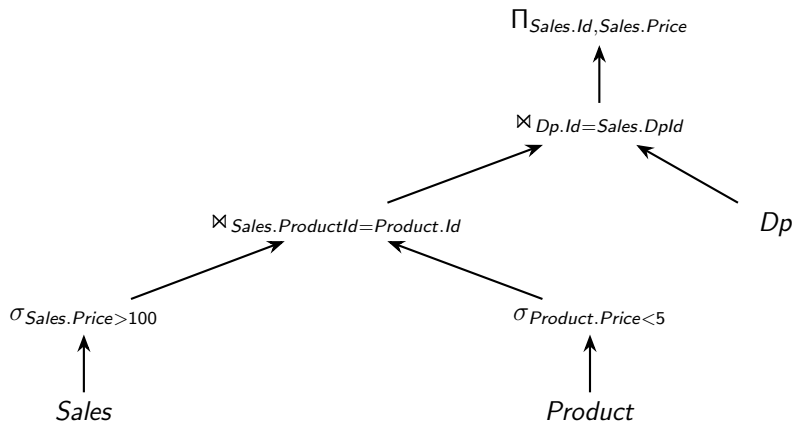


La descente des sélections

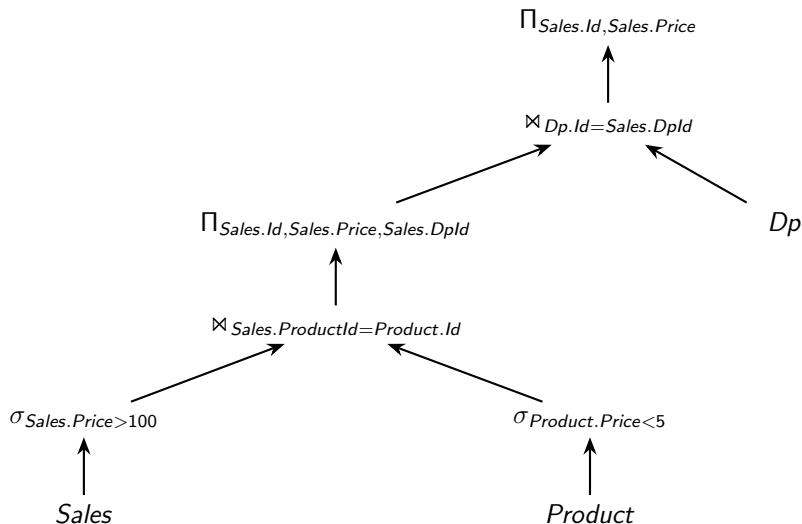


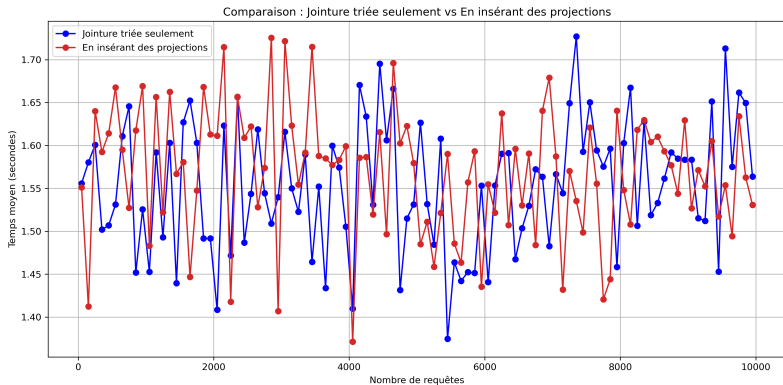


Cas problématique



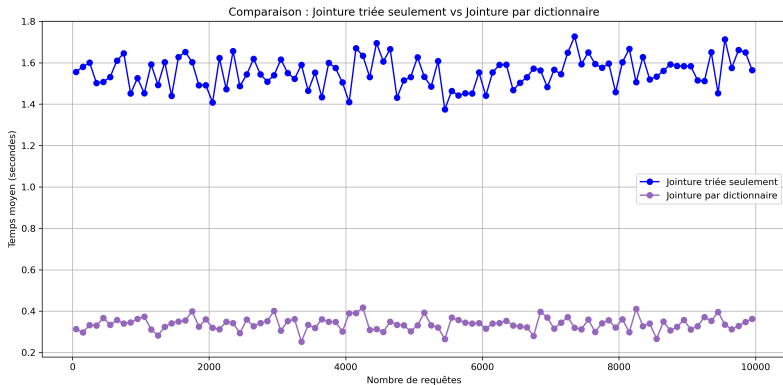
Insertion utile de la Projection





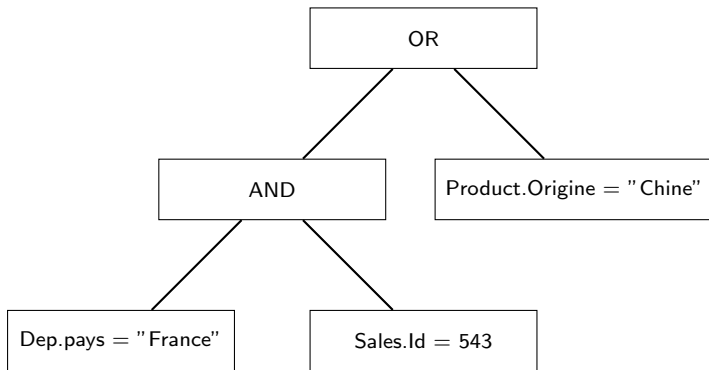
Les différents types de jointures possibles

- ▶ Produit Cartésien : $\mathcal{O}(n \times m)$
- ▶ Jointure Triée : $\mathcal{O}(n \log(n) + n + m \log(m) + m)$
- ▶ Jointure par dictionnaire : $\mathcal{O}(2n)$



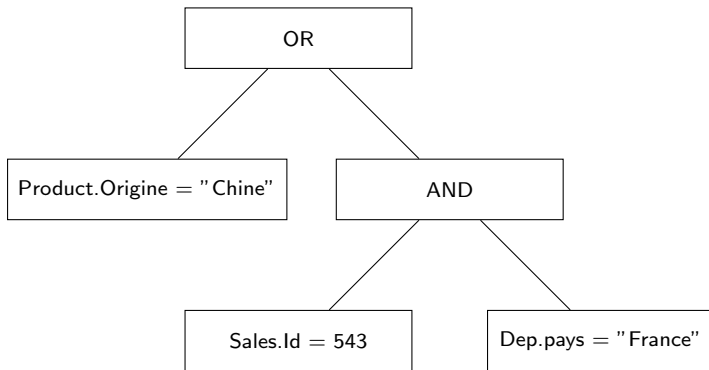
Analyse des expressions binaires

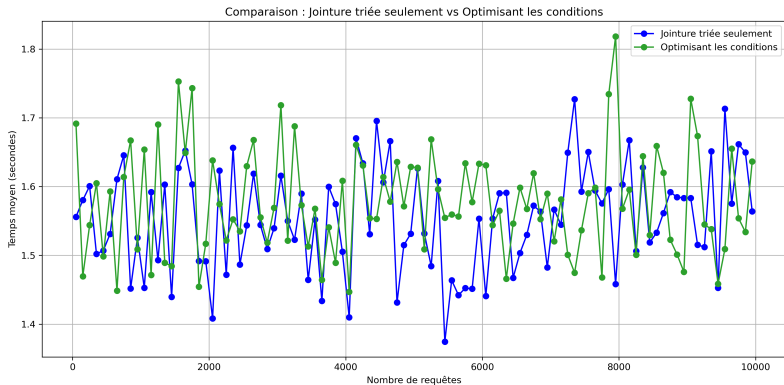
```
1 WHERE (Dep.pays = "France" AND Sales.Id = 543) OR (Product.  
    Origine = "Chine")
```



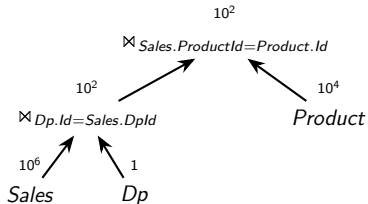
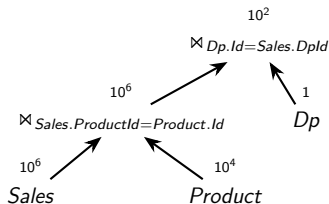
Optimisation des expressions binaires

```
1 WHERE (Dep.pays = "France" AND Sales.Id = 543) OR (Product.  
    Origine = "Chine")
```

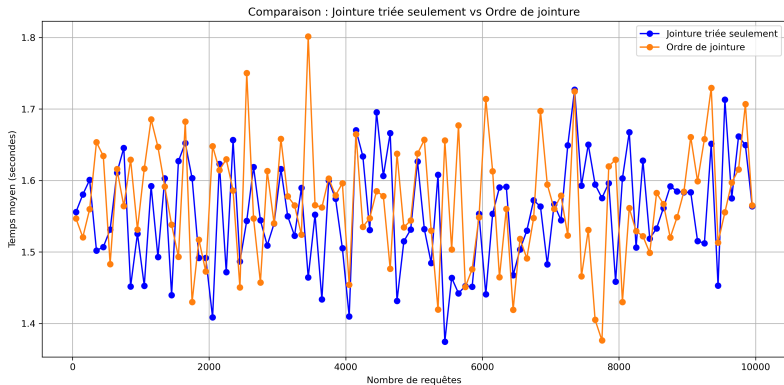




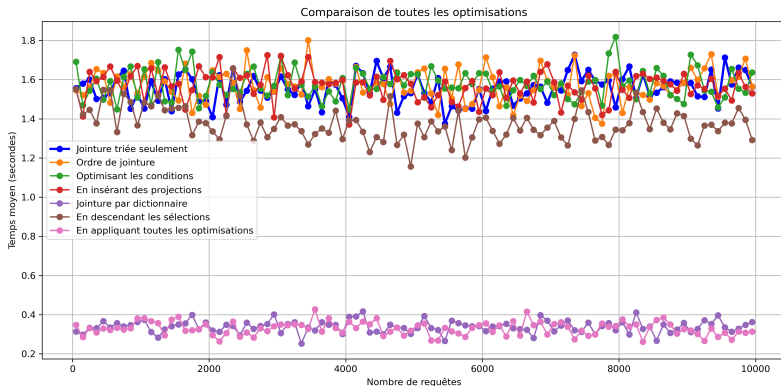
Résultats équivalents mais complexité différente



- ▶ *Sales.ProductId=Product.Id* d'abord → 10⁴ × 10⁶ + 10⁶ × 1 comparaisons
- ▶ *Dp.Id=Sales.DpId* d'abord → 1 × 10⁶ + 10² × 10⁴ comparaisons



Analyse de toutes les optimisations



Les raisons de la lenteur de notre modèle

- ▶ De nombreuses techniques n'ont pas été traitées, notamment les plus récentes (~ 20 ans)
- ▶ Les failles de notre code. 45 minutes $\rightarrow$ 22 secondes
- ▶ Les optimisations matérielles : Appel aux fonctions internes au processeur, le multi-threading
- ▶ La compression des données : *C-Store* 4 manière de compresser les données
- ▶ L'indexation des données facilitent les jointures

Résumé

- ▶ Différentes optimisations naïves utilisant l'algèbre relationnelle : Descente de sélection et insertion de projection
- ▶ Certaines optimisations dépendantes des données que notre SGBD possède

Ouverture

Les pistes récentes :

- ▶ Analyse des données $\Rightarrow$ Machine Learning (Génération de plan, Choix de l'ordre des jointures, ...)
- ▶ Une meilleure organisation des données en mémoire
- ▶ Les deux: *Hypothetical Index Benefit Estimation on Column-oriented Databases using Quantiles* 2025

Nos ressources

Nos ressources :

- ▶ *The Design and Implementation of Modern Column-Oriented Database Systems* 2013
- ▶ *C-Store: A Column-oriented DBMS* 2005
- ▶ *A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration* 2021
- ▶ *Extensible Query Optimizers in Practice* 2024
- ▶ *Foundation of Databases* 1995

Les bases de données orientées colonnes

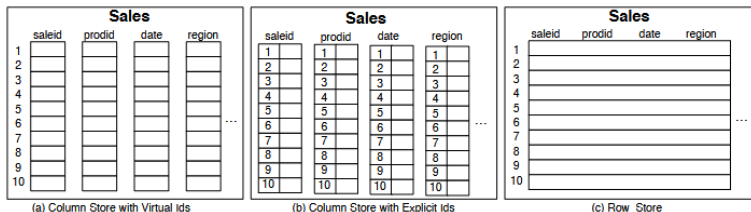
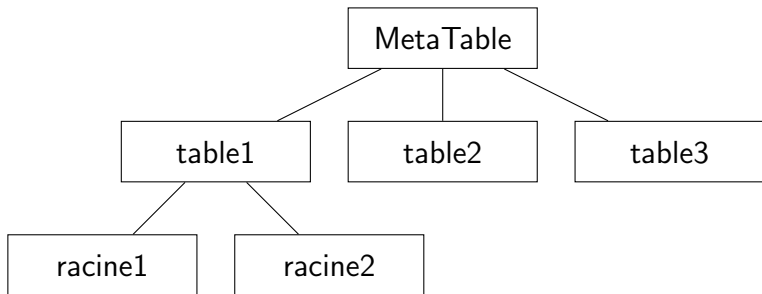


Figure: Rendu physique des BDD orientées colonnes et BDD orientées lignes

Notre gestion des tables durant l'exécution



- ▶ **Racine** : valeurs chargées.
- ▶ **Table** : liste de lignes encore valides.
- ▶ **MetaTable** : jointure de plusieurs *Tables*.
- ▶ **Particularité** : modifier les lignes d'une *Table* (jointure ou sélection) modifie toutes les *Tables*.